

[← Back to Home](#)

Exam Results

Session ID: hz2gvl6

[Share](#)[Challenge a friend](#)

✓ PASSED (MOCK THRESHOLD)

817 / 1000

Archetype: Tooling Architect

Excellent work! You've demonstrated a solid understanding of the architect blueprint.

[Download certificate](#)[Share on LinkedIn](#)**49**

CORRECT

**11**

INCORRECT

**0**

SKIPPED

**93m**

TIME TAKEN

Found this useful?

This mock is free and built by CyberSkill. A coffee keeps it running.

[☕ Buy me a coffee](#)

Building with Claude?

Building with Claude:

CyberSkill designs and ships Claude-powered agents for teams. This mock is one of our free resources.

[See our work](#)[Talk to us](#)**Performance by domain**

Research pipelines	13/15 (87%)
Extraction pipelines	14/15 (93%)
Customer support agents	12/15 (80%)
Code exploration	10/15 (67%)

Focus next on Code exploration.

Review Questions

Question 1**Correct**

Your agent has called `lookup\_order` multiple times while investigating a customer's return requests. Each response includes 40+ fields (items, shipping details, payment info, status history). Tool outputs now represent the majority of the conversation's context. The customer mentions two more orders they want to discuss. What's the most effective approach before making additional lookups?

A Extract only return-relevant fields (items, purchase date, return

window, status) from each existing order response, removing verbose details

Correct. Keep the fields that matter for the task and drop the rest. This directly addresses the context-bloat problem before you add two more lookups.

B Have the model generate a natural language summary of each order's key details, replacing structured responses with prose descriptions

Prose summaries lose precision for fields the model may need later (exact dates, amounts). Structured pruning is better than paraphrasing.

C Move all tool responses to a vector database with semantic indexing, retrieving relevant portions as the conversation continues

A vector DB is heavy infrastructure for what is essentially a pruning problem.

D Proceed with additional lookups without modifying the existing tool output context

Does nothing about the bloat – you're heading for context exhaustion.

Question 2

Correct

After adding an MCP server with specialized code refactoring tools (`extract_function`, `rename_variable`, `inline_function`), you notice the agent still uses basic text manipulation via Write and Bash sed commands for refactoring tasks. The MCP server is

connected and healthy. Examining the configuration, you find each MCP tool has a minimal description like "extract_function: extracts a function from code." What's the most effective way to improve adoption of the MCP refactoring tools?

- A** Implement a request classifier that detects refactoring intent and automatically routes those requests to the MCP server before the agent processes them.

A pre-classifier is a separate system to maintain and can misroute. The cheaper fix is to make the tool descriptions strong enough that the agent picks them on its own.

- B** Remove the Write tool from the agent's configuration for refactoring sessions so it must use the MCP tools for code modifications.

Stripping general tools forces adoption by subtraction, and breaks legitimate Write use cases.

- C** Accept this as expected behavior since simpler tools like sed are more predictable than specialized refactoring tools.

Capitulating defeats the purpose of integrating refactoring tools. The integration is fine – the descriptions are the problem.

- D** Enhance the MCP tool descriptions to explain when each tool is preferable to text manipulation and clarify expected inputs and outputs.

Correct. Tool selection is driven by the descriptions Claude sees. When the MCP tools say 'extracts a function from code' and Write/sed come with rich documentation, Claude picks Write/sed. Beef up the descriptions.

Question 3**Correct**

During testing, you observe that in extended exploration sessions (30+ minutes), the agent starts giving inconsistent answers about code structure it discussed earlier. Engineers report having to repeat context about modules they've already explored. What's the most effective approach to address this?

- A** Have the agent maintain a scratchpad file that records key findings, referencing it for subsequent questions.

Correct. A scratchpad offloads findings to durable storage the agent can re-read on demand, giving it a stable 'memory' independent of how crowded the context window gets.

- B** Switch to a higher-capacity model tier to provide more context window space for accumulated exploration data.

A bigger window delays the problem but doesn't solve attention degradation over long, noisy contexts.

- C** Implement automatic context clearing every 15 minutes to ensure the agent starts with fresh, uncontaminated context.

Wholesale clearing throws away valid findings – exactly the state engineers complain about having to re-establish.

- D** Create summaries of all source files before exploration begins, loading only these compressed representations into context.

Pre-summaries lose the detail that makes code exploration useful and often misrepresent files the engineer cares about most.

Question 4**Correct**

Production reviews reveal inconsistent handling of uncertainty in final reports. Sometimes conflicting subagent findings are synthesized into a single confident statement (losing nuance), while other times reports over-hedge with excessive qualifications (becoming unhelpful). When the web search agent returns "industry analysts estimate \$50B market size (methodology varies)" and the document analysis agent returns "peer-reviewed study estimates 35B($\pm$ 7B, 95% CI)," the coordinator either picks one arbitrarily or produces vague statements like "the market may be 35B–50B depending on factors." What systematic approach best addresses this?

- A** Configure subagents to only report findings meeting a high-confidence threshold, filtering uncertain information before it reaches the coordinator.

Filtering throws away useful-but-uncertain evidence. It doesn't help the synthesis agent reason about conflicts – it just hides them.

- B** Implement a confidence calibration layer that normalizes subagent uncertainty expressions to standardized probability scores (0.0–1.0), then weight-average findings by their calibrated confidence.

Collapsing peer-reviewed CIs and analyst estimates into a single averaged number destroys the methodological difference that's the whole point.

- C** Instruct the synthesis agent to structure reports with explicit

sections distinguishing well-established findings from contested ones, preserving original source characterizations and methodological context.

Correct. Report structure that keeps methodological context and separates settled vs. contested claims is how you get nuance without over-hedging.

- D Add a verification subagent that cross-references findings across sources, only passing claims to synthesis that are corroborated by at least two independent sources.

Two-source minimums drop legitimate single-source findings and don't resolve genuinely conflicting estimates with different methodologies.

Question 5

Correct

Your multi-agent research pipeline crashed after processing 12 of 28 documents. The web search agent had identified relevant sources, the document analysis agent had partially completed extraction, and the synthesizer had begun pattern identification. You need to resume processing without repeating work or losing fidelity of prior findings. What state management approach best balances information fidelity with context efficiency when restoring agent state?

- A Have each agent maintain its own persistent state file and reload it independently at the start of each session.

Fragmented per-agent state breaks coordinator visibility and makes cross-agent reasoning brittle. Each agent reloading unrelated internal state also bloats their context with noise they don't need.

- B** Persist the coordinator's conversation log containing all task delegations and responses, providing this to agents when resuming.

A full conversation log is the highest-fidelity but least context-efficient option. You'd replay everything into each subagent and blow past context budgets.

- C** Have each agent persist a structured report to a known location. On resume, the coordinator loads the reports and injects relevant state into agent prompts.

Correct. Structured per-agent reports keep fidelity (the findings, with schema), let the coordinator stay in charge of orchestration, and keep each subagent's context focused. This is the orchestrator + compact artifact pattern.

- D** Index all agent outputs in a shared vector store. When resuming, each agent queries the store using semantic search to retrieve relevant prior findings.

Semantic retrieval over agent outputs is overkill for resuming a crashed pipeline – it adds a retrieval failure mode and can miss precise state that a structured report preserves exactly.

Question 6

Correct

After the web search agent finds 25 sources (120K tokens of raw content), the document analysis agent extracts key insights (15K tokens), and the synthesis agent produces a coherent narrative draft (3K tokens), the coordinator must pass context to the report generation agent for the final output with proper source citations. What context-passing strategy provides the best balance of completeness and efficiency?

- A** Pass only the synthesis draft and have a separate post-processing pipeline match claims to sources and insert citations after the report is generated.

Post-hoc matching is fragile – without the mapping the model used, you can't reliably bind claims to the right source. Hallucinated or misattributed citations are the usual failure mode.

- B** Pass the synthesis draft along with a structured source index that maps key claims to their source URLs and relevant excerpts.

Correct. The synthesis gives the narrative; the source index gives the report generator exactly the binding it needs to cite without re-reading 120K tokens of raw content.

- C** Pass a condensed summary of all prior stages that preserves the main findings and attributes them to sources by name only.

Name-only attribution loses URLs and excerpts, so the report generator can't quote or verify – and 'by name only' tends to drift into vague citations.

- D** Pass the full accumulated context from all prior agents.

Maximum completeness but wasteful – 120K+ tokens of raw search content is mostly irrelevant noise at the report-generation stage.

Question 7

Correct

After investigating a billing dispute over 25+ turns, you've identified that duplicate charges occurred due to a payment

gateway timeout triggering retry logic. The required refund (\$847) exceeds your \$500 authorization limit. You need to call `escalate\_to\_human`, and the human agent won't have access to your conversation transcript. What context should you pass to enable effective resolution?

- A** The customer's original complaint verbatim plus the tool result excerpts showing duplicate transactions.

Raw artifacts without synthesis force the human to re-do the 25 turns of investigation you just finished.

- B** A structured summary: customer ID, root cause, refund amount, and recommended action.

Correct. A structured handoff with identifiers, cause, amount, and recommended action is what a human agent needs to pick up the case instantly without re-investigating.

- C** The complete conversation transcript with all tool results.

Dumping the whole transcript forces the human to wade through 25 turns instead of reading a one-screen brief.

- D** Your diagnosis and the refund amount only.

Missing customer identifiers and recommended action – the human can't act without them.

Question 8

Correct

An engineer asks the agent to find all callers of a function before

removing it. The function is defined in a core library but is also exposed through wrapper modules that rename the function for domain-specific use (e.g., `calculateTax` in the library becomes `computeOrderTax` in the orders module). What exploration strategy will most reliably identify all callers?

- A** Read the library and wrapper modules to identify all exposed names for the function, then Grep for each name across the codebase.

Correct. You have to enumerate every name the function is exposed under – otherwise renamed wrappers hide callers. Read the relevant modules, gather all aliases, then grep for each.

- B** Use Grep to find all files that import from the library or wrapper modules, then read each file to check whether it uses the function.

Misses dynamic imports, re-exports, and indirect call chains. Also scales poorly.

- C** Use Grep to search for the function's original name across the codebase.

The original name won't appear anywhere a wrapper has renamed it – you'd miss a significant class of callers.

- D** Search for the function name in project documentation to understand intended usage patterns and navigate to documented integration points.

Docs are incomplete and often stale. You can't safely remove a function based on documentation-level evidence.

Question 9**Correct**

An engineer used the agent yesterday to analyze a legacy authentication module, identifying two distinct refactoring approaches: extracting a microservice versus refactoring in-place. Today, they want to explore both approaches in depth—having the agent propose specific code changes for each—before deciding which to implement. What's the most effective way to structure this exploration?

- A** Resume yesterday's session to explore the first approach, then start a new session for the second, manually recreating the original context.

Manual recreation is error-prone and loses the exact working state of yesterday's analysis.

- B** Start two fresh sessions, manually providing a summary of yesterday's analysis findings to establish context.

Redoes work and risks the two sessions diverging from the same baseline you established yesterday.

- C** Resume yesterday's session and explore both approaches sequentially within the same conversation thread.

Sequential exploration in one thread lets each approach contaminate the other's context.

- D** Use ``fork_session`` to create two branches from yesterday's analysis, exploring one approach in each fork.

Correct. Forking from yesterday's session gives each approach its own independent context starting from the same analysis baseline – clean, parallel, no contamination.

Question 10

Correct

Your agent is handling a billing dispute. After calling ``get_customer`` and ``lookup_order``, it identifies that the dispute involves a promotional pricing error requiring manager approval – beyond the agent's authorization level. How should the workflow handle this mid-process escalation?

- A Call ``escalate_to_human`` passing only the customer's original message.

Drops the tool-derived context the agent just gathered, forcing the human to re-do the investigation.

- B Compile a structured handoff with customer details, order info, and the identified issue before calling ``escalate_to_human``.

Correct. A structured brief (who, what order, what issue, why it exceeds auth) lets the human agent pick up instantly. That's the mid-process escalation pattern.

- C Attempt the refund with ``process_refund`` anyway, escalating only if the system rejects the transaction.

Knowingly exceeding authorization is a policy violation – not something to try and hope the system catches.

- D Persist the complete conversation and tool response history to

- Persist the complete conversation and tool response history to a database, then call `escalate\_to\_human` with a reference ID.

Adds infrastructure and an extra lookup step when an inline structured brief is simpler and faster.

Question 11

Correct

The web search agent has gathered several relevant sources for a research topic. The document analysis agent now needs to examine these sources. How does information typically flow between these two specialized subagents?

- A The agents communicate through an event-driven message queue, with the document analysis agent subscribing to web search completion events.

Event buses aren't part of the Claude subagent model. Subagents don't publish/subscribe to each other directly.

- B The web search agent directly invokes the document analysis agent, passing the discovered sources as parameters.

Subagents are isolated – they can't directly call sibling subagents. That would also tightly couple them and defeat the orchestrator pattern.

- C The coordinator agent receives the web search agent's output and includes relevant findings in the prompt when invoking the document analysis agent.

Correct. In an orchestrator-worker pattern the coordinator is the hub. It collects each subagent's output and explicitly forwards the relevant parts into the next subagent's prompt.

- D Both agents access a shared memory store where the web search agent writes findings and the document analysis agent reads them.

A shared store can be layered in as an optimization, but it's not how subagents typically communicate – and it introduces stale-read and consistency problems.

Question 12

Correct

After your daily batch of 10,000 documents completes, 300 documents (3%) failed with "context_length_exceeded" errors. The results file identifies each failure by `custom\_id`. What's the most cost-effective approach to process these failures?

- A Reprocess the entire batch with prompt caching enabled to reduce the cost of retrying requests with identical system prompts

Re-running 10,000 successful documents to solve 300 failures is wasteful regardless of caching savings. And caching doesn't fix the length-exceeded error.

- B Resubmit only the 300 failed documents after chunking them into smaller pieces, then combine the partial extractions

Correct. Targeted, and it addresses the actual cause (input too long). Chunk the oversized docs, extract per chunk, then merge – minimum tokens, fixes the specific failure mode.

- C Resubmit the entire 10,000 document batch using a model tier

with a larger context window

Expensive (reprocessing 9,700 successes) and may still fail on truly outsized documents.

- D Increase the ``max_tokens`` parameter for the 300 failed documents and resubmit them in a new batch

``max_tokens`` controls output length, not input length. It doesn't solve ``context_length_exceeded``, which is about the combined input-plus-output budget.

Question 13

Correct

Your system has been operating with 100% human review for 3 months. Analysis shows that extractions with model confidence >90% have 97% accuracy overall. To reduce reviewer workload, you plan to automate high-confidence extractions. Before deploying, what validation step is most critical?

- A Analyze accuracy by document type and field to verify high-confidence extractions perform consistently across all segments, not just in aggregate.

Correct. Aggregate accuracy hides segment failures – one document type could be 70% while others are 99%. Auto-routing by overall number alone risks systemic errors in the weak segments.

- B Compare accuracy at different confidence thresholds (85%, 90%, 95%) to find the optimal cutoff that maximizes automation while minimizing errors.

Threshold tuning is worth doing, but only after you know the accuracy

holds uniformly across segments – otherwise you're optimizing a number that lies.

- C Run a two-week pilot routing 25% of high-confidence

Claude

Certified

Architect

MOCK
EXAM – BY
CYBERSKILL

Practice

Leaderboard

Dashboard

Guide

About

Support



- D Verify that 97% accuracy meets requirements for all downstream systems that consume the extracted data.

An important product question, but it assumes the 97% holds everywhere – again, that's what segment analysis verifies first.

Question 14

Correct

Your extraction pipeline processes restaurant menus and must output structured JSON with fields for item names, descriptions, prices, and dietary tags. Some menus use inconsistent formatting—prices as "\$12" vs "12.00", dietary info as icons vs text. What's the most reliable approach?

- A Use separate extraction calls for each field to ensure consistent handling of each type.

Multiple calls per menu multiplies cost and latency and loses cross-field context (e.g., which dietary icons sit next to which item).

- B Extract data as-is and normalize formats in post-processing

code after Claude returns.

Post-processing a raw blob is brittle – deterministic code has to handle every format permutation. Better to let the model normalize at extraction time.

- C** Request multiple extraction attempts per document and select the most common format.

Ensemble voting is expensive and can still pick a wrong-format majority.

- D** Define a strict output schema and include format normalization rules in your prompt.

Correct. Strict schema + explicit normalization rules ('prices as decimal with two places', 'dietary as enumerated tags') lets the model both extract and normalize in one pass.

Question 15

Correct

A customer returns 4 hours after their initial session about the same billing dispute. The previous 32-turn session contains 'lookup_order' results showing "Status: PENDING, Expected resolution: 24-48 hours." In testing, you observe that when resuming sessions with stale tool results, the agent often references the outdated data in responses (e.g., "I see your refund is still being processed") even after subsequent fresh tool calls return different information. What approach most reliably handles returning customers?

- A** Resume with full history but filter out previous 'tool_result' messages before resuming, keeping only the human/assistant

turns so the agent must re-fetch needed data.

Stripping `tool\_results` from the middle of a conversation can leave assistant messages referencing nonexistent results – the transcript becomes internally inconsistent.

- B** Start a new session, inject a structured summary of the previous interaction (issue type, actions taken, resolution status), then make fresh tool calls before engaging.

Correct. A clean session with a summary keeps the narrative continuity while guaranteeing the agent isn't reasoning over stale tool results.

- C** Resume with full history and add a system prompt instruction telling the agent to always prefer the most recent tool results when multiple calls to the same tool exist in context.

Prompt instructions are suggestions. You observed that exact failure mode in testing – the model still references old results.

- D** Resume with full history and configure the agent to automatically re-call all previously-used tools at session start to ensure data freshness.

Blanket re-calling is wasteful, slow, and still leaves the old results sitting in context to confuse the model.

Question 16

Incorrect

Documents arrive continuously throughout business hours and need structured data extracted. To reduce costs, you want to use the `Message Batches API` (50% discount, up-to-24-hour processing window). Your SLA specifies that extraction results

must be available within 30 hours of document arrival with 99.9% reliability. Which batching strategy is most appropriate?

- A** Submit batches every 6 hours containing documents from that window

Worst-case a document waits 6 hours to be batched and then up to 24 hours to process = 30 hours exactly. No safety margin for a 99.9% reliability target.

- B** Submit a single batch at end of day containing all documents from that day

A doc arriving just after morning startup waits all day to be batched plus up to 24 hours to process – well over 30 hours.

- C** Submit batches every 4 hours containing documents from that window

Correct. Max 4-hour wait + up to 24-hour batch SLO = 28-hour worst case, leaving a 2-hour cushion under the 30-hour SLA to absorb batch variance and hit 99.9%.

- D** Use the real-time API for all documents instead of batch processing

Meets SLA trivially but throws away the 50% cost saving the question asks you to capture.

Question 17

Correct

Your agent needs to insert a new helper function into the middle of a 150-line utility module, between two existing functions. The

Edit tool fails because its ``old_string`` parameter cannot find unique text to match – the file has repetitive docstrings, variable names, and structural patterns. What's the most reliable way to complete this insertion?

- A** Use Edit with an extremely long ``old_string`` capturing 30+ lines of context to guarantee uniqueness

Long, brittle match strings frequently miss due to whitespace or minor edits and produce confusing failures.

- B** Use Edit's ``replace_all`` parameter to target a common pattern and embed the new function in the replacement text

``replace_all`` would mutate every occurrence of the pattern – corrupting the whole file.

- C** Use Bash to append the function definition to the end of the file using heredoc syntax

Appending puts the function at the wrong location. The requirement is to insert between two existing functions.

- D** Use Read to load the file, add the function at the appropriate location, then Write the updated file

Correct. When Edit's unique-match contract can't be satisfied in a repetitive file, fall back to Read → modify in memory at the intended line → Write the full file back.

Question 18

Incorrect

A customer raises three separate issues during one session: a refund inquiry (turns 1–15), a subscription question (turns 16–30), and a payment method update (turns 31–45). At turn 48, the customer asks "What happened with my refund?" The conversation is approaching context limits. What strategy best maintains the agent's ability to address all issues throughout the session?

- A** Extract and persist structured issue data (order IDs, amounts, statuses) into a separate context layer.

A sidecar context layer adds engineering and doesn't naturally preserve the conversational narrative the customer expects.

- B** Rely on MCP tools to re-fetch relevant information on demand when the customer references earlier issues.

Re-fetching is fine for freshness, but it doesn't solve the context-length problem – and you lose what was said between the customer and the agent.

- C** Summarize earlier turns into a narrative description, preserving full message history only for the active issue.

Correct. Progressive summarization compresses stable resolved topics while keeping the active thread verbatim – the classic pattern for long multi-issue conversations near the context limit.

- D** Implement sliding window context that retains the most recent 30 turns.

A pure sliding window silently drops the refund issue (turns 1–15) from context, which is exactly what the customer is asking about.

Question 19**Correct**

Your extraction system parses e-commerce product descriptions to extract specifications like dimensions, weight, and materials into JSON. Despite having a well-defined schema, the model inconsistently extracts the "materials" field—sometimes returning "cotton blend", other times "Cotton/Polyester mix", and occasionally omitting the field when material information is clearly present in the source. What's the most effective way to improve extraction consistency?

- A** Make the "materials" field required instead of optional in the schema to force the model to always extract a value

Required-ness doesn't solve format inconsistency, and it can force invented values when material info is genuinely missing.

- B** Switch to a more capable model tier since inconsistent extraction indicates insufficient model capability

Costly, and doesn't address the actual cause — the model doesn't know the canonical output format you want.

- C** Set temperature to 0 to eliminate randomness and ensure deterministic outputs

Determinism doesn't define the right format; it just makes the wrong-format answers consistent.

- D** Add few-shot examples showing 2-3 complete input-output pairs with standardized material description formats

Correct. Few-shot examples demonstrate the exact canonical format

you want ('cotton/polyester' as a normalized list), and they also raise recall on material info that was being skipped.

Question 20

Incorrect

A customer sends: "This is frustrating. I've explained my issue twice and nothing is being resolved. I want to talk to a real person NOW." The agent has not yet called any tools to investigate their account. What should the agent do?

- A** Acknowledge the frustration and ask one targeted question to understand the specific issue before escalating.

Correct. The customer has said 'twice' but you have no context yet. One acknowledging, focused question gives you a shot at first-contact resolution without dismissing the frustration or delaying a potential handoff.

- B** Briefly explain what the agent can help with and offer to resolve the issue quickly, escalating only if the customer repeats their request.

Launching into capability-listing at a frustrated customer who asked for a human reads as dismissive.

- C** Immediately call `escalate\_to\_human` with the conversation history.

Escalating with zero tool context creates a cold handoff where the human also starts from scratch.

- D** First call `get\_customer` and `lookup\_order` to gather account

context, then escalate to a human agent.

Investigating without asking adds latency and doesn't respect the customer's request. A single acknowledging question is faster and more respectful.

Question 21

Correct

Your schema includes a `skills: string[]` field. Production monitoring reveals three consistency issues: (1) compound phrases like "Python and SQL" are sometimes kept as one entry, sometimes split; (2) implied but unstated skills occasionally appear in extractions; (3) similar documents produce wildly different array lengths (5-10 vs 40+ entries). Your prompt currently says "Extract all skills mentioned." What's the most effective improvement?

- A** Add few-shot examples demonstrating compound phrase handling, explicit mention criteria, and appropriate entry granularity.

Correct. All three issues are about the model's interpretation of what counts as 'a skill.' Few-shot examples teach the pattern concretely – split vs. not-split, mentioned vs. inferred, appropriate granularity.

- B** Add constraints: "Extract 10-20 skills maximum, one skill per entry, only explicitly named skills."

Hard count caps are arbitrary and can force the model to drop real skills or invent filler to hit the range.

- C** Add post-extraction normalization that maps skills to a

canonical taxonomy and deduplicates similar entries.

Post-processing can't fix inferred-but-not-mentioned skills or choose the right split for 'Python and SQL' – that decision has to be made at extraction time.

- D Enrich the schema to `{skill: string, confidence: float, 'source_quote': string}[]` to capture extraction metadata.

Useful signal, but doesn't address the underlying inconsistency in how skills are parsed in the first place.

Question 22

Correct

When analyzing complex legal cases that cite multiple precedents, the document analysis subagent processes each sequentially. A landmark case citing 12 precedents takes over 3 minutes to analyze completely. What's the most effective way to reduce this latency while preserving the coordinator's ability to monitor and debug the system?

- A Implement a message queue where precedent analysis tasks are processed asynchronously by a pool of worker agents.

External queues complicate observability – the coordinator loses direct visibility into which tasks succeeded and what they produced.

- B Create a recursive agent hierarchy where analysis agents subdivide work among child agents until reaching single-precedent granularity.

Recursion adds levels of indirection that make debugging and monitoring harder, for no real speedup beyond the first fan-out.

- C** Have the coordinator spawn parallel document analysis subagents, each handling a subset of precedents, then aggregate results before synthesis.

Correct. Coordinator-managed parallelism fans out the work, keeps each subagent's scope tight, and preserves a single hub for monitoring and aggregation.

- D** Enable the document analysis subagent to spawn its own specialized subagents dynamically when it encounters cases with many citations.

Nested spawning hides execution inside subagents and makes the coordinator's debug view incomplete.

Question 23

Correct

An engineer asks your agent to identify untested code paths in a legacy payment processing module spanning 45 files. After reading the first 8 source files, the agent's responses are becoming noticeably less accurate—it's forgetting previously discussed code patterns and hasn't yet located all test files or traced critical payment flows. What's the most effective approach to complete this investigation?

- A** Document all current findings in a summary report, clear context completely, then use that report as the sole reference for continuing the investigation.

A single report becomes the only source of truth and tends to compress away the specific code patterns you'd need later.

- B** Spawn subagents to investigate specific questions (e.g., "find all test files for payment processing", "trace refund flow dependencies") while the main agent coordinates findings and preserves high-level understanding.

Correct. Delegate well-scoped investigations to subagents with fresh context, while the main agent keeps the architectural overview. This is the pattern for scaling exploration beyond a single context window.

- C** Clear context with /clear, then selectively re-read only the most critical files discovered so far, writing key findings to a scratchpad file that persists between context resets.

Scratchpads help, but clearing + re-reading discards the understanding you already built across 8 files.

- D** Switch to using Grep to search for specific function names instead of reading full files, reducing the content loaded into context for remaining exploration.

You can't identify untested paths just by grepping names – you need to read enough of each path to know what branches the tests don't cover.

Question 24

Correct

Compliance requires that refunds exceeding \$500 must automatically escalate to a human agent – this rule cannot be left to model discretion. Despite clear system prompt instructions, production logs show the agent occasionally processes high-value refunds directly (3% failure rate). How should you achieve guaranteed compliance?

- A** Modify the refund tool to return an error with message "Amount exceeds policy limit – please escalate" when threshold is exceeded.

This helps, but depends on the agent interpreting the error correctly and escalating – still model discretion at the decision point.

- B** Add few-shot examples to the prompt showing correct escalation behavior at various refund amounts (\$400, \$500, \$600).

Few-shots shift the distribution but don't remove the 3% failure. Compliance says the rule can't be left to model discretion.

- C** Implement a hook to intercept tool calls; when the refund process amount exceeds \$500, block it and invoke human escalation.

Correct. Compliance-grade rules belong outside the model – a deterministic hook on the tool call is guaranteed to fire every time, independent of model behavior.

- D** Strengthen the system prompt with emphatic language: "CRITICAL POLICY: Refunds over \$500 MUST trigger human escalation. NEVER process these directly."

Emphatic prompts reduce but don't eliminate misuse. 'Cannot be left to model discretion' rules out any prompt-only solution.

Question 25

Correct

A customer writes: "I've been going back and forth on this return for days. I just want to speak to someone who can actually help

me." The agent has confirmed via `lookup\_order` that the return is straightforward—within policy and eligible for immediate processing. What should the agent do?

- A** Acknowledge frustration, inform them this is resolvable now, and offer to complete it or escalate

Correct. Honor the feeling, give them the fast resolution path in writing, and preserve their choice. That's the customer-respect move that still leverages the agent's capability.

- B** Call `escalate\_to\_human` immediately to honor the customer's request

Unnecessary queueing when the issue is one tool call away. Frustrates the customer further by adding waiting to an already-simple case.

- C** Process the refund via `process\_refund` to resolve the underlying issue, then inform them it's complete

Takes unilateral action after the customer explicitly asked to speak to someone – overrides their stated preference.

- D** Ask what specifically hasn't worked in previous attempts before deciding whether to escalate or resolve automatically

Interrogating a frustrated customer about past failures is the opposite of what they asked for.

Question 26

Correct

Your extraction system implements automatic retries when

validation fails. On each retry, the specific validation error is appended to the prompt. This retry-with-error-feedback approach resolves most failures within 2-3 attempts. For which failure pattern would additional retries be LEAST effective?

- A The model extracts keywords as a nested object organized by category when the schema requires a flat array of strings

A structural mismatch the model can definitely correct given feedback about the expected shape.

- B The model extracts citation counts as locale-formatted strings ("1,234") when the schema requires integers

The required number is present in the document – the model just needs to stop formatting it. Easy for feedback-retry to fix.

- C The model extracts dates as ISO 8601 datetime strings ("2023-03-15T00:00:00Z") when the schema requires only the date portion (YYYY-MM-DD)

Pure formatting fix – the underlying value is correct and the model can trim the time portion on retry.

- D The model extracts "et al." for co-authors when the full list exists only in an external document not in the input

Correct. No amount of retrying teaches the model information that isn't in the input. Retry-with-error-feedback only fixes mistakes the model could have gotten right from the source.

Question 27

Correct

An engineer used `Claude Code` yesterday to investigate authentication flows in a legacy monolith, building up significant context over a 2-hour session. Today she wants to continue that specific investigation. She's worked on three other codebases since then and knows the session was named "auth-deep-dive". How should she resume?

A Start fresh and re-read the same files

Wastes the two hours of accumulated context from yesterday.

B Use `--session-id` with the UUID from yesterday's session transcript file

Possible but cumbersome – she'd have to hunt for the UUID when she already has the session name.

C Use `--continue` to pick up where the most recent conversation left off

`--continue` resumes the most recent session in this repo. She's worked on three other codebases since then, so the most recent is not auth-deep-dive.

D Use `--resume` auth-deep-dive to load that specific session by name

Correct. `--resume` with the session name is designed for exactly this: pick a specific prior session out of many, by the name you gave it.

Question 28

Correct

When implementing your `lookup\_order` MCP tool, the backend sometimes returns errors (e.g., "Order not found" or temporary database failures). What is the correct pattern for communicating these errors back to the agent?

- A** Log the error server-side and return an empty result to avoid confusing the model

Returning empty successes makes the agent think no data exists, rather than that something went wrong – a different and worse confusion.

- B** Return the error message in the tool result content with the `isError` flag set to true

Correct. MCP's designed pattern: put the error text in the content field and mark `isError=true`. Claude sees both the failure flag and a readable message to reason about.

- C** Throw an exception from the tool handler so the agent framework can catch and log it

Uncaught exceptions break the tool protocol and don't give the model anything to reason with.

- D** Return a success response with a "status" field indicating the error type

Ad-hoc 'status' fields vary across tools and the model has no standard way to interpret them. `isError` is the standard.

Question 29

Correct

A developer asks the agent to investigate why a specific API endpoint intermittently returns 500 errors. The codebase has 200+ files and the developer doesn't know which components are involved. The agent must trace the error through routing, middleware, business logic, and database layers. What task decomposition approach would be most effective?

- A** Have the agent first create a comprehensive plan mapping all code paths through the endpoint before beginning any file exploration or code reading.

You can't build a correct plan for an unknown error without any exploration. Planning blind wastes time and misses the actual failure path.

- B** Have the agent dynamically generate investigation subtasks based on what it discovers at each step, adapting its exploration plan as new information about the error path emerges.

Correct. Debugging is adaptive by nature – each file you read changes the most useful next step. Let the agent follow the evidence.

- C** Define a fixed sequence of investigation steps upfront—grep for error patterns, then read error handlers, then check database queries, then examine middleware—executing each step regardless of intermediate findings.

A fixed pipeline wastes work on layers that aren't involved and can lock the agent out of the actual root cause path.

- D** Run parallel worker agents that simultaneously investigate all four layers, then synthesize their findings to identify where the error originates.

Fan-out is useful once you have well-scoped subtasks. Here, you don't – you'd pay 4× the cost to look in three layers that aren't the problem.

Question 30

Correct

Production logs reveal inconsistent error handling: when `lookup_order` fails, the agent sometimes retries 5+ times (wasteful when the order ID doesn't exist), sometimes escalates immediately (premature for temporary network issues), and sometimes asks users for clarification (inappropriate when the issue is a backend permission error). Investigation shows your MCP tool returns uniform error responses: `{"isError": true, "content": [{"type": "text", "text": "Operation failed"}]}`. The agent cannot distinguish between error types. What's the most effective improvement?

- A Enhance error responses with structured metadata: include `errorCategory` (transient/validation/permission), `isRetryable` boolean, and a description of what caused the failure.

Correct. Give the agent the information it needs to make the right decision: category, retryability, and a human-readable cause. That replaces guessing with deterministic policy.

- B Create an `analyze_error` MCP tool the agent calls after any failure to determine the error category and recommended action.

Adds an extra round-trip for something the original tool already knows. Put the metadata in the original response.

- C** Implement retry logic with exponential backoff in your MCP server for all errors, returning to the agent only after retries are exhausted.

Blanket retries hurt on permanent errors (order not found) and hide useful distinctions from the agent.

- D** Add few-shot examples to the system prompt demonstrating how to interpret error message patterns and select appropriate responses for each.

If the tool returns 'Operation failed' for every failure, no amount of few-shots can extract category info that isn't there.

Question 31

Incorrect

Your codebase exploration tool stores session IDs to allow engineers to continue investigations across work sessions. An engineer spent an hour yesterday analyzing a legacy authentication module, building context about its architecture and dependencies. They want to continue today. The session ID is valid, but version control shows 3 of the 12 files the agent previously read were modified overnight by a teammate's merge. What approach best balances efficiency and accuracy?

- A** Resume the session without informing the agent about the changed files

Silent resume leaves the agent reasoning on stale content for 3 of 12 files – exactly the source of bad recommendations.

- B** Start a fresh session to ensure the agent works with current

codebase state without stale assumptions

Throws away an hour of valid context about 9 files that didn't change.

- C** Resume the session and inform the agent which specific files changed for targeted re-analysis

Correct. Keeps the expensive context you already built, while telling the agent exactly which 3 files to re-read – minimum waste, maximum accuracy.

- D** Resume the session and immediately have the agent re-read all 12 previously analyzed files

Unnecessary for the 9 unchanged files. Just adds tokens without improving accuracy.

Question 32**Correct**

In production, you observe that simple fact-checking queries (e.g., "What year was the Paris Climate Agreement signed?") traverse all four subagents sequentially, consuming 40+ seconds and significant tokens per query. Complex comparative research benefits from the full pipeline. Your query distribution is diverse and evolving as users discover new applications. What's the most effective approach to optimize for varying query complexity?

- A** Implement pattern-based routing that categorizes queries by structure (single-fact vs. comparative vs. analytical) and maps each category to a predefined subagent combination.

Pattern routing calcifies as the query distribution evolves – new

intents break the patterns and silently land on the wrong pipeline.

- B** Create a fast-path for factual questions that bypasses subagents entirely, routing all other queries through the complete pipeline to ensure research thoroughness.

A binary fast-path fixes the worst case but wastes the full pipeline on every moderately complex query that could have used a subset.

- C** Have the coordinator analyze each query and dynamically decide which subagents to invoke based on its assessment of query requirements.

Correct. Letting the coordinator LLM reason about each query and pick only the subagents it needs adapts naturally to an evolving, diverse query distribution – this is the strength of the orchestrator pattern.

- D** Train a query complexity classifier on labeled historical data to predict optimal subagent combinations, retraining periodically as query patterns evolve.

A trained classifier needs labels, retraining, and drift monitoring. That's over-engineered when the coordinator can make the same call at runtime from the query itself.

Question 33

Correct

The coordinator provides detailed step-by-step instructions to the web search subagent, specifying exact search queries, source priorities, and date filters. Production monitoring reveals three issues: (1) the subagent reports "insufficient results" rather than trying alternative approaches when pre-specified searches

fail, (2) research quality drops for emerging topics that don't match expected patterns, and (3) the subagent rarely surfaces valuable tangential sources. What's the most effective way to improve subagent adaptability?

- A** Remove procedural details entirely, delegating with simple goals like "research X thoroughly" and relying on the subagent's general capabilities.

Too far in the other direction – 'research X thoroughly' loses the guardrails (source quality, recency) that make delegation reliable.

- B** Add explicit fallback directives to the detailed instructions: "If specified searches yield fewer than N results, attempt alternative query formulations before reporting failure."

Patches one failure mode but keeps the agent locked into procedural thinking – still brittle for emerging topics and tangential sources.

- C** Implement a topic classification step where the coordinator categorizes requests as "well-defined" or "exploratory" and uses different instruction styles for each category.

A classifier with two buckets is fragile, and you're still shipping rigid instructions in the well-defined branch.

- D** Specify research goals and quality criteria (coverage breadth, source diversity, recency) rather than procedural steps, letting the subagent determine its search strategy.

Correct. Delegate intent and quality bars, not procedures. The subagent can then choose queries, follow promising tangents, and recover from dead ends on its own.

Question 34**Correct**

In production, final reports frequently contain claims without proper source attribution. Investigation shows that while the web search and document analysis agents correctly attach citations to their outputs, the synthesis agent loses track of which sources support which conclusions when combining findings. What's the most effective architectural change?

- A** Maintain complete transcripts of all subagent interactions and add a citation-resolution agent to analyze logs and determine attributions before report generation.

Post-hoc log analysis is fragile and expensive – and still loses bindings when synthesis merges points from multiple sources.

- B** Require all subagents to output structured claim-source mappings that the synthesis agent must preserve and merge when combining findings from multiple sources.

Correct. Explicit claim-source mappings are a first-class output the synthesis agent can merge deterministically – no attribution gets dropped during summarization.

- C** Add a verification step where the report generator uses semantic similarity matching against original sources to reconstruct which claims came from which documents.

Similarity search can attribute the wrong source when two sources say similar things. You want the model's original mapping, not a reconstruction.

- D** Have the coordinator inject source identifier prefixes into text

before each handoff, then parse these prefixes at report generation to reconstruct citations.

Prefix tokens inside prose get dropped, paraphrased, or hallucinated. Structured mappings outside the prose are more robust.

Question 35

Correct

The agent verifies customer identity through a multi-step process before resetting passwords. During testing, you notice that after the customer answers the third verification question, the agent asks them to provide their name again, as if the earlier exchange never happened. What's the most likely cause of this behavior?

- A The verification tool is clearing the agent's internal state after each successful validation step.

Tools don't clear agent state – conversation state lives in the messages array you send.

- B The prompt lacks instructions telling Claude to remember information across multiple exchanges.

Memory across turns isn't achieved by an instruction – it comes from passing the prior turns into the next request.

- C The conversation history isn't being passed in subsequent API requests.

Correct. The API is stateless. Each request must include the full messages array. If you only send the latest turn, the model has no

memory of earlier ones – exactly the 'ask for the name again' symptom.

- D Claude's memory retention is limited to two conversational turns by default, requiring explicit configuration to extend it.

No such default limit exists. Context is bounded by the window, and history length is under your control.

Question 36

Correct

Your extraction pipeline processes contracts that frequently include amendments. When a contract contains both original terms and later amendments (e.g., original clause specifies "30-day payment terms" while Amendment 1 changes this to "45 days"), the model inconsistently extracts one value or the other with no indication of which applies. What's the most effective approach to improve extraction accuracy for documents with amendments?

- A Redesign the schema so amended fields capture multiple values, each with source location and effective date.

Correct. Amendments are structurally about versioned values. A schema with value + source location + effective date models the domain correctly and stops forcing the model to pick one.

- B Add prompt instructions to always extract the most recent amendment value and ignore superseded original terms.

Downstream consumers sometimes need the original too (e.g., for dispute resolution). Hard-coding 'most recent wins' throws that away.

- C** Preprocess documents with a classifier that identifies and removes superseded sections before the main extraction step.

Deleting sections with a classifier can accidentally strip content that's still legally effective – amendments often modify subsets of clauses.

- D** Implement post-extraction validation using pattern matching to detect amendments and flag those extractions for manual review.

Routes every amended contract to a human – expensive, and still doesn't give downstream systems a structured answer.

Question 37

Correct

You're implementing the escalation logic for when the agent should call ``escalate_to_human``. Your team proposes four different approaches for triggering escalation. Which approach will most reliably identify cases that genuinely require human intervention?

- A** Instruct the agent to escalate when the customer requests a human, when the issue requires policy exceptions, or when the agent cannot make meaningful progress.

Correct. Escalation decisions are judgment calls about intent and progress – exactly what LLMs are good at. Clear criteria in natural language outperform rigid rules for the long tail.

- B** Configure the agent to escalate after three consecutive tool calls that fail to resolve the customer's stated issue, ensuring a reasonable attempt before involving a human.

A hard retry count fires both too early (legitimate retries) and too late (obvious policy issues on the first call).

- C** Implement sentiment analysis that monitors for frustration indicators (negative language, repeated questions, exclamation marks) and trigger escalation when the frustration score exceeds a configured threshold.

Sentiment can catch frustration but misses calm customers who simply need a human for a policy exception, and over-escalates on stylistic language.

- D** Build a rules engine that maps specific issue types, customer segments, and product categories to escalation decisions, removing the need for model judgment calls.

Rules engines break on the cases they weren't designed for – and customer support is full of those.

Question 38

Correct

Your system extracts event metadata (date, location, organizer, `attendee\_count`) from news articles using a JSON schema with all nullable fields. During evaluation, you observe the model frequently generates plausible but incorrect values for fields not mentioned in the article—for example, outputting "500" for `attendee\_count` when the source contains no attendance information. What's the most effective way to reduce these false extractions?

- A** Add a post-processing step using a second LLM call to verify

each extracted value exists in the source document.

Expensive second pass for a behavior you can correct at the source – tell the model to return null when the field isn't stated.

- B** Add prompt instructions to return null for any field where information is not directly stated in the source.

Correct. The fields are already nullable; the model just needs an explicit instruction to prefer null over a plausible guess. This is the standard fix for schema-aware hallucination.

- C** Make all schema fields required (non-nullable) with strict validation rules to ensure the model only outputs verifiable data.

Required fields force the model to invent values when information is missing – the opposite of what you want.

- D** Upgrade to a more capable model tier with improved instruction-following to reduce hallucination tendencies.

Adds cost and still doesn't tell the model your policy – returning null instead of inventing.

Question 39

Correct

Your extraction uses tool use with a JSON schema where `property\_type` is defined as an enum: ['house', 'apartment', 'condo', 'townhouse']. After deployment, 8% of extractions fail schema validation. Investigation reveals listings mention many uncommon property types—"studio", "loft", "duplex", "mobile

home", "tiny house", "converted warehouse"—and new types continue appearing regularly. What's the most effective long-term solution?

- A** Continuously expand the enum to include newly observed property types and add monitoring for additional edge cases.

Whack-a-mole enum maintenance — every new listing style risks another validation failure.

- B** Add an "other" value to your enum with a separate ``property_type_detail`` string field for specifics when "other" is selected.

Correct. Keeps the strong enum for the common cases (clean downstream joins) while giving a well-typed escape hatch that preserves detail. Stable long-term.

- C** Change ``property_type`` from an enum to a free-form string and implement a normalization step in post-processing.

Loses the validation guarantee entirely and pushes the normalization problem downstream with no canonical vocabulary.

- D** Add few-shot examples to your prompt demonstrating how to map unexpected property types to the closest existing enum value.

Forcing 'tiny house' into 'house' drops real distinctions downstream — you're silently lossy.

Question 40

Correct

After integrating a local MCP server providing code analysis

After integrating a local MCP server providing code analysis tools (`analyze\_dependencies`, `find\_dead\_code`, `calculate\_complexity`), you verify the server is healthy and tools appear in the tools/list response. However, you observe that the agent consistently uses Grep to search for import statements instead of calling `analyze\_dependencies`—even when users explicitly ask about "code dependencies." Examining tool definitions reveals: MCP: `analyze\_dependencies` – "Analyzes dependency graph" Built-in: Grep – "Search file contents for a pattern using regular expressions. Returns matching lines with line numbers and surrounding context." What's the most effective approach to improve the agent's selection of MCP tools?

- A** Remove Grep from available tools when the MCP server is connected to eliminate functional overlap.

Crippling a general tool to force adoption of a specialized one punishes other legitimate Grep use cases.

- B** Add routing instructions to the system prompt specifying that dependency-related questions should use MCP tools rather than Grep.

Prompt-level routing can help, but the root cause is that the MCP tool's description is thinner than Grep's. Fix the tool descriptions first.

- C** Split `analyze\_dependencies` into granular tools (`list\_imports`, `resolve\_transitive\_deps`, `detect\_circular\_deps`) so each has a focused purpose less likely to overlap with Grep.

Splitting is sometimes right, but here the sibling tools would still have weak descriptions. The same selection bug would reappear.

- D** Expand MCP tool descriptions to detail capabilities and outputs –e.g., "Builds dependency graph showing direct imports, transitive dependencies, and cycles."

Correct. Tool selection is driven by the descriptions the model sees. A one-line description like 'Analyzes dependency graph' loses against Grep's rich description. Beef up the MCP tool's description.

Question 41

Correct

After implementing tool use with strict schema definitions, JSON syntax errors are eliminated, but 5% of extractions still have valid JSON with empty arrays or null values for required fields like citations and methodology. Spot-checking reveals that source documents contain this information, but in varied formats—inline citations vs. bibliographies, methodology sections vs. details embedded in introductions. What's the most effective way to address these failures?

- A** Implement retry logic that re-sends requests when validation detects empty required fields.

Retry on the same prompt won't suddenly teach the model to recognize methodology embedded in an intro. The extraction capability is the gap.

- B** Build a regex-based post-processing layer that scans source documents for citation patterns and methodology keywords, populating empty fields when the model fails to extract.

Regex over prose is brittle, misses the nuanced cases (intro-embedded methodology), and ships silently wrong data.

- C** Modify your schema to make citations and methodology optional, and flag incomplete records for manual review rather than failing validation.

Papers over the symptom. The fields really are required downstream – routing them to humans trades one problem for reviewer workload.

- D** Add few-shot examples demonstrating extractions from documents with varied structures—showing how to identify citations in different formats and locate methodology details across section types.

Correct. The failure mode is the model not recognizing varied formats. Concrete examples across the format distribution directly raise recall on the 5%.

Question 42

Incorrect

The document analysis agent has a single ``analyze_document`` tool that takes a document and a free-text instruction parameter. During evaluation, requests like "extract the key financial metrics" often return narrative summaries, while "summarize the methodology" sometimes returns raw data tables. The synthesis agent reports that 35% of analysis results require re-requests with clarified instructions. What's the most effective way to improve reliability?

- A** Split the generic tool into purpose-specific tools—``extract_data_points``, ``summarize_content``, ``verify_claim_against_source``—each with defined input/output contracts.

Correct. Free-text instructions put the semantics in prose, which the

model interprets inconsistently. Purpose-specific tools give the model an explicit, well-typed contract to pick between.

- B** Keep the single tool but add an ``analysis_type`` enum parameter requiring explicit selection between extraction, summarization, and verification modes.

Better than free text, but still a single tool with a single output shape. Output still has to be a generic string, and the model can mode-mismatch. Separate tools enforce distinct I/O contracts.

- C** Have the coordinator pre-classify each analysis request before passing instructions to the document analysis agent.

Moves the ambiguity upstream without fixing it. The tool contract is still fuzzy; the coordinator's classification is one more error surface.

- D** Enhance the tool description with detailed examples showing how different instruction phrasings should map to different output formats.

Examples help, but tool-use reliability comes first from clear tool boundaries and output schemas, not from heroic prompt engineering on one generic tool.

Question 43

Correct

A user is expanding the research system beyond its single web search agent by adding specialized data sources. They add a financial API agent that returns structured JSON with revenue, margins, and growth rates; a news monitoring agent that returns prose summaries of recent developments; and a patent analysis agent that returns structured lists of technology areas. The

synthesis agent combines these into executive briefings. Currently, it converts everything to bullet points, causing financial comparisons to lose tabular clarity and news summaries to lose narrative flow. What change would most improve briefing quality?

- A** Standardize all subagent outputs to prose summaries with inline citations.

Flattening structured financial data into prose loses the comparability you want in an executive briefing.

- B** Add a format conversion layer between subagents and synthesis that transforms all outputs to a common intermediate representation.

A single IR usually ends up too lossy – either it keeps structure (news suffers) or it keeps prose (tables suffer). The fix is type-aware rendering, not uniformity.

- C** Update the synthesis agent to render each content type appropriately – financial data as tables, news as prose.

Correct. Executive briefings need mixed rendering: tables for numbers, prose for narrative. Asking synthesis to preserve the native format of each input is the right abstraction.

- D** Standardize all subagent outputs to JSON with fields for claim, evidence, source, and confidence.

Forcing prose-heavy news into a claim/evidence schema strips the narrative flow executives actually read.

Question 44**Correct**

An engineer asks the agent to understand how the caching layer works before adding a new cache invalidation trigger. After initial Grep searches, the agent has identified that caching logic spans 15 files including decorators, middleware, and service classes (~8,000 lines total). What's the most effective next step for building understanding while managing context constraints?

- A** Use the Read tool to sequentially load all 15 files, building complete understanding across the full caching implementation.

Loading 8,000 lines without structure floods the context and buries the invalidation logic in unrelated code.

- B** Analyze imports and class hierarchies to identify the base cache class, Read that file to understand the interface, then trace specific invalidation implementations.

Correct. Start from the architectural root (the interface), then navigate only the specific implementations that matter for invalidation – focused reading, low context cost.

- C** Use Grep to search for "invalidate" and "expire" patterns across all files, then Read only those specific line ranges with minimal surrounding context.

Keyword-only reading strips the class/method context that tells you what the invalidation actually guards.

- D** Use Glob to find files matching common caching patterns (cache.py, caching/), prioritize the largest files by reading them

first, then check smaller files for gaps.

Size is a poor proxy for importance. You'd waste context on big utility files and miss the small strategic ones.

Question 45

Incorrect

An engineer's exploration subagent spent 30 minutes analyzing a legacy payment system, reading 47 files and documenting data flows. The session was interrupted when the engineer's connection dropped. While away, a teammate merged a PR that renamed two utility functions. The engineer wants to continue the same exploration. What's the most effective approach?

- A Resume the subagent from its previous transcript without mentioning the changes—the architecture understanding remains valid.

Silently resuming lets the agent keep referencing the old function names in its recommendations.

- B Launch a fresh subagent and include the prior transcript in the initial prompt for context.

Loading 30 minutes of transcript into a new subagent's prompt is wasteful and pollutes its starting context.

- C Launch a fresh subagent with a summary of prior findings.

Re-summarization throws away the fine-grained data flows the agent was already tracking across 47 files.

- D Resume the subagent from its previous transcript and inform it

about the renamed functions.

Correct. Keep the accumulated understanding, and give it a targeted delta about the renames so it can update its mental model – minimum waste, maximum accuracy.

Question 46

Correct

Your extraction system processes two document types: standard monthly reports (archived after processing) and urgent exception reports (must trigger business alerts within 30 minutes of receipt). Both use the same JSON schema. You want to minimize API costs while meeting latency requirements. How should you architect the processing pipeline?

- A Submit all documents to the real-time Messages API to ensure consistent processing latency across document types.

Consistent but expensive. Standard monthly reports don't need real-time latency and shouldn't pay for it.

- B Submit all documents to the 'Batch API' with 'custom_ids' for tracking. When results arrive, immediately process urgent documents and trigger delayed alerts for exceptions.

Batch has up to a 24-hour window – 30-minute alerting SLAs on exception reports can't be met by routing them through batch.

- C Queue all documents and submit hourly batches, flagging urgent documents for expedited handling when batch results return.

Hourly batches still ride the batch SLO and don't guarantee the 30-

...many batches can hit the batch API and don't guarantee the 50-minute alert window for exceptions.

- D** Route standard reports to the 'Batch API' for 50% cost savings, and route urgent exception reports to the real-time Messages API.

Correct. Match latency profile to document urgency: batch for the bulk (cheap), real-time for the latency-sensitive exceptions (fast). Minimizes cost while meeting SLA.

Question 47

Correct

When the agent calls 'lookup_order' and receives order details showing the item was purchased 45 days ago, how does the agentic loop determine whether to call 'process_refund' or 'escalate_to_human' next?

- A** The orchestration layer automatically routes to the next tool based on the order's status field.

There's no implicit orchestration layer picking tools from a field. The model drives tool selection.

- B** The agent follows a pre-configured decision tree mapping order attributes to specific tool calls.

Agentic loops are model-driven, not decision-tree-driven. Hardcoded trees are the opposite of what the agent pattern is for.

- C** The order details are added to the conversation and the model reasons about which action to take.

Correct. The agentic loop works by appending `tool_result` messages to the conversation and letting the model decide the next step on each turn. That's how 45 days → refund vs. escalate gets resolved.

- D The agent executes the remaining steps in a tool sequence planned at the start of the request.

There's no upfront, committed plan – the agent chooses each next step based on the latest context.

Question 48

Correct

After deployment, you find that 12% of extractions contain semantic errors that pass JSON schema validation (e.g., a duration like "30 minutes" incorrectly placed in an ingredient quantity field). Human reviewers have capacity to check only 20% of extractions. Which approach most effectively allocates reviewer attention?

- A Have the model output field-level confidence scores, then calibrate review thresholds using a labeled validation set.

Correct. Field-level confidence lets you route the low-confidence 20% – which is where the 12% semantic errors concentrate – to humans. Calibration makes the threshold choice data-driven.

- B Randomly sample 20% of extractions for review, using corrections to track accuracy and identify error patterns.

Great for measurement, poor for coverage – random sampling only catches 20% of the 12% errors (~2.4% total). Confidence-based routing finds far more.

- C** Prioritize review of all extractions where required fields are empty or explicitly marked as not found.

Empty fields are a narrow error mode. The 12% problem is wrong values in populated fields, which this filter misses.

- D** Review all extractions from documents with formatting anomalies such as unusual layouts or mixed content types.

A useful heuristic, but formatting anomaly doesn't reliably predict where fields got confused with each other.

Question 49

Incorrect

Your agent has analyzed a complex service module—reading 23 source files, tracing request flows, and identifying error handling patterns. A developer wants to compare two testing strategies before committing to one: end-to-end tests with mocked external services vs. snapshot tests capturing expected outputs. They need to independently develop both approaches to evaluate trade-offs. How should you manage the sessions?

- A** Export the analysis session's key findings to a file, then create two new sessions that reference this file.

An exported summary is lossier than the live session state the agent built up while reading 23 files.

- B** Resume the analysis session with `'fork_session'` enabled, creating a separate branch for each testing strategy.

Correct. Forking gives each strategy its own independent context starting from the exact analysis baseline – no cross-contamination, no re-analysis.

- C** Start two fresh sessions, having each re-read the relevant source files before beginning.

Burns tokens redoing work the original session already did, and risks each fresh session reaching different conclusions from the same code.

- D** Continue in the original session, developing end-to-end tests first, then snapshot tests sequentially.

Sequential development in one thread lets the first strategy's implementation bias reasoning about the second.

Question 50

Incorrect

An engineer who just joined the team asks the agent to help them understand the authentication and authorization architecture before making security improvements. The codebase has 800+ files across multiple services. What exploration strategy will most effectively build understanding, given Claude built-in tools and context limits?

- A** Read any CLAUDE.md and README files first, then ask the engineer to specify which 10-15 files are most important for understanding the auth system.

The engineer just joined – they probably don't know which files matter. That's exactly what the agent is supposed to help with.

- B** Launch parallel subagents to explore different services simultaneously, then synthesize their findings into an architectural overview.

Without knowing where auth lives, fan-out explores too broadly and subagents duplicate and miss cross-service flows.

- C** Use Grep to find authentication entry points, read those files, then follow imports and function calls to map the auth flow incrementally.

Correct. Start at entry points (login, token verify, middleware), then trace outward following real code edges. Incremental, grounded, fits within context limits.

- D** Read all files containing "auth", "login", "permission", or "token" in their content or filename.

Those keywords hit a huge amount of unrelated code across 800+ files and drown context in noise.

Question 51

Correct

The coordinator agent has `AgentDefinitions` configured for all four specialized subagents, each with appropriate descriptions, prompts, and tool restrictions. During testing, you notice the coordinator correctly reasons about when to delegate—it generates messages like "I'll ask the web search agent to find sources on this topic"—but no subagent execution ever occurs. The coordinator then proceeds as if the delegation happened and continues with incomplete information. Logs show no errors. What is the most likely cause?

- A The coordinator's `max_tokens` setting is too low, causing the Task tool invocation to be truncated before the subagent type parameter can be specified.

A truncation would show up in logs and usually leave partial `tool_use` blocks – not silent no-ops.

- B The `AgentDefinitions` are configured correctly, but the coordinator's system prompt doesn't explicitly list the available subagent types, preventing the model from knowing they can be invoked.

Tool/agent schemas are surfaced to the model automatically – you don't need to re-list them in the system prompt for the model to see them.

- C The coordinator's `allowedTools` configuration doesn't include "Task", so while it can reason about delegation, it cannot invoke the tool required to spawn subagents.

Correct. Without the Task tool in `allowedTools`, the coordinator can talk about delegating but has no way to actually call a subagent – which matches the 'reasons about it, no execution, no errors' symptom.

- D Subagent context isolation means task descriptions from the coordinator don't automatically reach subagents; you need to configure explicit context forwarding in `ClaudeAgentOptions`.

Context isolation is real, but it affects what the subagent sees once spawned – not whether the coordinator can spawn it at all.

Question 52

Correct

During a billing dispute resolution, your agent successfully retrieves customer info via ``get_customer`` and order details via ``lookup_order``, but when attempting to call ``process_refund``, the tool returns a timeout error. The agent has enough information to explain the charges and verify refund eligibility, but cannot actually process the refund due to the backend failure. What approach best balances first-contact resolution with appropriate error handling?

- A** Escalate immediately to a human agent since the refund action cannot be completed

Punts a transient backend issue to a human when the agent could still explain the situation and offer options.

- B** Implement automatic retries with exponential backoff for ``process_refund``, keeping the conversation open until the refund is successfully processed

Forcing the customer to wait on indefinite retries is poor UX and may never succeed if the outage is long-lived.

- C** Explain the billing, confirm refund eligibility, acknowledge the system issue preventing immediate processing, and offer escalation or retry later

Correct. Deliver the partial value you can (explanation + eligibility), be honest about the failure, and let the customer choose between human escalation or a retry. Classic graceful degradation.

- D** Confirm the refund will be processed and close the conversation, since the system has all necessary information to complete it automatically

Committing to an outcome that didn't happen is misleading the

customer – a bigger failure than the timeout.

Question 53

Incorrect

Your ``process_refund`` tool returns two types of errors: technical errors ("503 Service Unavailable", "Connection timeout") that are transient (5% of calls), and business errors ("Order exceeds 30-day return window", "Item already refunded") that are permanent (12% of calls). Monitoring shows the agent wastes 3-4 turns retrying business errors that can never succeed. Currently, both error types return only a plain text message to Claude. What's the most effective way to reduce wasted retries while improving customer-facing response quality?

- A Return structured error responses with `retryable: false` for business errors and a customer-friendly explanation for Claude to use.

Correct. A `retryable` flag tells Claude deterministically 'don't retry,' and a ready-made customer-friendly message improves the outgoing reply. Fixes both problems at once.

- B Add few-shot examples showing how to distinguish `retryable` from non-`retryable` errors by parsing error message text.

Relying on the model to text-parse error strings is fragile and exactly the instability you see today.

- C Add a ``check_refund_eligibility`` tool that must be called before ``process_refund`` to prevent business rule violations.

Useful in principle but adds a round-trip to every refund to guard

against 12% of cases, and doesn't help when `process\_refund` still fails for other business reasons.

- D** Implement automatic retry logic at the tool level for technical errors only, passing business errors to Claude without retries.

Hiding transient retries inside the tool can mask latency and takes the model out of the loop on recovery decisions. Business errors also still arrive as a plain string, so the customer-facing response doesn't improve.

Question 54

Correct

Your extraction pipeline processes invoices and extracts line items, subtotals, tax amounts, and grand totals. During evaluation, you discover that in 18% of extractions, the sum of extracted line item amounts doesn't match the extracted grand total—sometimes due to OCR errors in the source document, sometimes due to extraction mistakes by the model. Downstream accounting systems reject records with mismatched totals. What's the most effective approach to improve extraction reliability?

- A** Add a "calculated_total" field where the model sums extracted line items alongside a "stated_total" field. Flag records for human review when values differ.

Correct. Capturing both values makes the discrepancy a first-class signal – you catch OCR errors and extraction mistakes uniformly, and you can route only the mismatched 18% to humans.

- B** Extract line items and totals independently, then use a separate validation model to reconcile discrepancies by determining

which extracted values are most likely correct.

A reconciliation model can mask OCR errors by silently picking one number over another – the accounting system still gets wrong data without a flag.

- C** Add few-shot examples demonstrating invoices where extracted line items sum correctly to the stated total, encouraging the model to produce mathematically consistent extractions.

Can nudge the model toward consistency, but when the source itself is internally inconsistent (OCR errors), the model has to choose one 'wrong'.

- D** Implement post-processing that automatically adjusts line item amounts proportionally when their sum doesn't match the stated total.

Silent financial data rewrites are dangerous – you'd be fabricating line items for a downstream accounting system.

Question 55

Correct

The synthesis agent receives summarized findings from the web search and document analysis agents, then passes a consolidated summary to the report generator. During testing, you discover the generated reports make factual claims without proper citations—the report generator cannot attribute statements to their original sources because that metadata was lost during the summarization steps. What's the most effective approach to ensure proper source attribution in the final reports?

- A** Have each agent output structured data separating content summaries from source metadata (URLs, document names, page numbers).

Correct. Structured content + separate source metadata preserves the mapping end-to-end, so the report generator receives both what was said and where it came from.

- B** Have the report generator query the web search agent to re-locate sources for claims in the final report.

Re-querying is slow, lossy, and invites misattribution – the web search agent doesn't know which claim came from which source.

- C** Instruct the synthesis agent to embed source references inline within its summary text using a consistent citation format.

Inline citations in free text drift and get dropped during later summarization. Structured metadata survives transformations better.

- D** Skip summarization and pass full raw outputs from web search and document analysis directly to the report generator.

You'd blow the context budget and make the report generator's job harder. Summarization is useful; losing metadata during summarization is the actual bug.

Question 56

Correct

After the web search agent and document analysis agent complete their tasks, the coordinator invokes the synthesis agent. However, the synthesis agent responds that it cannot

complete the task because no research findings were provided.
What is the most likely cause of this issue?

- A The synthesis agent's context window is not large enough to hold the combined outputs from both previous agents.

A context-window overflow typically surfaces as a truncation or API error, not as the agent saying 'no findings provided.'

- B The coordinator did not include the outputs from the previous agents in the synthesis agent's prompt.

Correct. Subagent invocations are isolated – nothing flows between them unless the coordinator explicitly puts it in the prompt. The message 'no findings provided' is exactly what you'd see.

- C The subagents need to share a single API connection to enable automatic context sharing between invocations.

There's no 'automatic context sharing' over a shared connection. Context isolation is by design.

- D The synthesis agent needs tools that can fetch results directly from the other agents' conversation histories.

Cross-agent history fetching isn't a standard capability and isn't the right fix – the coordinator should be forwarding findings explicitly.

Question 57

Incorrect

Your agent has spent 25 minutes exploring a game engine's rendering subsystem—reading shader code, buffer management,

and frame synchronization logic. An engineer now asks it to understand how the physics engine integrates with rendering for collision debug overlays. You notice recent responses reference "typical rendering patterns" rather than the specific VulkanPipeline and FrameGraph classes it discovered earlier. What's the most effective approach?

- A** Spawn a sub-agent to explore physics independently, then manually synthesize its findings with the rendering knowledge accumulated in the main conversation.

Manual synthesis in the main conversation pushes even more into an already-degrading context.

- B** Continue in the current context with more targeted prompts referencing the specific classes by name.

The agent is already losing specificity ('typical patterns' instead of VulkanPipeline). Pushing harder on the same degraded context doesn't restore it.

- C** Summarize key rendering findings, then spawn a sub-agent for physics exploration with that summary in its initial context.

Correct. Condense what you've learned about rendering into a compact summary, then give a fresh subagent that summary plus the physics task – you preserve the important signal and escape the degraded context.

- D** Use /clear to reset context completely, then start fresh with physics exploration using file paths from the project's CLAUDE.md.

/clear nukes the rendering findings you actually need for the cross-cut question about collision debug overlays.

Question 58**Correct**

Your pipeline uses a tool called ``extract_metadata`` with a JSON schema for paper details. You've also defined ``lookup_citations`` and ``verify_doi`` tools for enrichment. During testing, you notice that when users include requests like "extract the metadata and tell me how cited it is," Claude sometimes calls ``lookup_citations`` first, which fails because it needs the DOI that ``extract_metadata`` would provide. What's the most effective way to ensure structured metadata extraction happens first?

- A Set ``tool_choice`` to "any" so Claude must use a tool, combined with system prompt instructions prioritizing ``extract_metadata``.

'any' forces a tool call but doesn't force the right one. You'd still see ``lookup_citations`` picked first.

- B Set ``tool_choice`` to "auto" and reorder the tool definitions so ``extract_metadata`` appears first in the tools array, since Claude prioritizes earlier-listed tools.

There's no documented ordering preference to rely on; this assumes behavior that isn't contractual.

- C Set ``tool_choice`` to `{"type": "tool", "name": "`extract_metadata`"}` and process the enrichment requests in subsequent turns after receiving the extracted metadata.

Correct. ``tool_choice`=specific-tool` deterministically forces ``extract_metadata`` on the first turn. Then you hand control back to 'auto' to let the model use citations/DOI enrichment with the metadata

in context.

- D Set `tool_choice` to `{"type": "tool", "name": "extract_metadata"}` for every API call in the pipeline, ensuring Claude always extracts metadata before any enrichment can occur.

Pinning `extract_metadata` on every call prevents the enrichment tools from ever being used.

Question 59

Correct

When researching "renewable energy adoption," the web search agent returns recent statistics (2024: 35% adoption) while the document analysis agent extracts data from internal reports (2022: 18% adoption). The synthesis agent incorrectly flags these as contradictory sources rather than recognizing the data shows growth over time. What change would best enable the synthesis agent to correctly interpret such temporal differences?

- A Require subagents to include publication or data collection dates in their structured outputs.

Correct. The synthesis agent misreads the data because it never sees the dates. Making each data point carry its own timestamp in the structured output lets synthesis reason about trends instead of contradictions.

- B Add a conflict resolution agent that automatically discards older data when newer data exists for the same metric.

Silently discarding older data destroys trend information – the very thing the question is asking about.

- C** Configure the web search agent to only return results from the past 6 months.

Shrinking the window throws away historical context and doesn't fix the architectural gap that metadata isn't being passed to synthesis.

- D** Instruct the synthesis agent to always treat the most recent data as authoritative and place older findings in a separate historical appendix.

Prompt instructions alone are unreliable and still hide temporal reasoning behind a rule. Structuring the data with dates is the systematic fix.

Question 60

Incorrect

Production monitoring shows that follow-up queries like "summarize what we learned about market trends" consistently take 40+ seconds. Investigation reveals the coordinator spawns the synthesis subagent for each summarization request, passing 80K+ tokens of accumulated findings. The coordinator already has these findings in its context from orchestrating the research. What's the most effective way to improve response time for these follow-up summaries?

- A** Pre-generate and cache summaries at multiple granularities whenever new findings accumulate.

Speculative generation wastes tokens for summaries the user may never ask for, and cache granularities never quite match what's asked.

- R** Have the coordinator handle straightforward summarization

- B** Have the coordinator handle straightforward summarization requests directly using its existing context, reserving subagent spawning for complex analysis.

Correct. If the coordinator already has the findings, spawning a subagent to re-ingest 80K tokens is pure overhead. Let the coordinator answer simple follow-ups itself.

- C** Enable prompt caching on the synthesis subagent to reduce the overhead of repeatedly transferring the same research findings.

Prompt caching trims cost on the repeated prefix but the architectural waste – spawning an entire subagent for a summary – remains.

- D** Spawn the synthesis subagent with reduced context and have it request specific findings from the coordinator on-demand.

Adds round-trips and complexity for the same result the coordinator could produce directly.



CyberSkill

Turn Your Will Into
Real

A software solutions consultancy building products and developer tools. This mock exam is one of our free resources for the Claude practitioner community.

MOCK EXAM

Practice
Leaderboard
Your dashboard
Changelog
About

LEARN

Study guide
Sample questions
FAQ
Domains

CYBERSKILL

🌐 cyberskill.world
✉ info@cyberskill.world
👉 Buy me a coffee



Support this

project

© 2026 CyberSkill Software Solutions Consultancy and
Development JSC. Ho Chi Minh City, Vietnam.

Unofficial study aid – not affiliated with or
endorsed by Anthropic.